

Exercices de programmation en Ocaml

préparation à l'agrégation d'informatique

Ces exercices sont à réaliser en Ocaml. Utilisez un interpréteur pour exécuter les fonctions demandées et conservez une liste de tests bien choisis pour pouvoir les réexécuter plus tard.

Apportez le plus grand soin à la **lisibilité** de votre code. Pour cela, il est essentiel de choisir des **noms de variables et de fonctions** qui aident le lecteur à comprendre le code. Respectez aussi les règles d'**indentation**.

Exercice 1. Réécrire les fonctions 1 à 4 du précédent TD à l'aide des fonctions `iter`, `iteri`, `map`, `mapi`, `fold_left`, `fold_right` des modules `List` et `Array` de OCaml.

Exercice 2. Définir une fonction qui incrémente toutes les valeurs d'un tableau d'entiers.

Exercice 3. Définir une fonction `itere` qui calcule la n -ième itérée d'une fonction: (`itere f n`) `x` doit retourner le même résultat que (`f (f ... (f x) ...)`).

Exercice 4. Définir une fonction `teste_preds` qui évalue une liste de prédicats sur une valeur donnée.

Exercice 5. Définir une fonction `sum` qui permette ensuite d'écrire des sommes du type `sum 1 100 (fun i -> i * i)` pour se rapprocher de la notation mathématique $\sum_{i=1}^{100} i^2$.

Exercice 6. Écrire une fonction de type `'a -> (unit -> 'a)`. Proposer une utilisation de cette fonction.

Exercice 7. Implémenter une structure de liste doublement chaînée avec les opérations usuelles.

Exercice 8. Implémenter une structure de données permettant d'effectuer en temps $O(\log n)$ les opérations suivantes:

- insérer un élément,
- extraire l'élément de plus petite clé,
- extraire l'élément de plus grande clé.

Indication: vous utiliserez un tas min et un tas max contenant les mêmes éléments. La correspondance entre les deux tas sera assurée par des pointeurs.